Reviewer Pack — Littlewood-Richardson Coefficient Gap in Permanent vs Determ…

Entry 52ba9a0b4010 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-01 08:38:08 UTC

Littlewood-Richardson Coefficient Gap in Permanent vs Determinant Decompositions

Entry ID: `52ba9a0b4010` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-01 08:38:08 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl duality **Field B** (complexity object): Homogeneous polynomial complexity

Statement:

> For all n ≥ 2, the sum of Littlewood-Richardson coefficients for the decomposition of perm_n^⊗k into irreps λ ⊢ n^2 has Θ(n^{k-1}) growth, while det_m^⊗k has Θ(n^{k}) growth for m = ⌊n^{1.5}⌋.

Rationale (proposer's reasoning):

> Schur-Weyl duality decomposes tensor powers into irreducible representations, revealing structural asymmetries between permanent and determinant. The conjecture posits that the growth rate of specific coefficient sums reflects inherent complexity gaps detectable via representation-theoretic invariants.

Taxonomy category: `GCT_DET_PERM` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `41bcd94b0b29e6fa`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM_1	LLM_2
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.5s

5.1 Generated Python source

```python
import random
from math import factorial, floor

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def young_tableaux(n):
        if n == 0:
            return [[]]
        tableaux = []
        for i in range(1, n + 1):
            for subtableau in young_tableaux(n - i):
                tableaux.append([i] + subtableau)
        return tableaux

    def hook_length_formula(tableau):
        n = len(tableau)
        h = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                h[i][j] = (n - i) + (n - j) - 1
        det = 1
        for i in range(n):
            for j in range(n):
                det *= tableaux[i][j]
                det //= h[i][j]
        return det

    def littlewood_richardson_coefficient(λ, μ):
        if len(λ) != len(μ):
            return 0
        n = len(λ)
        a = [0] * (n + 1)
        b = [0] * (n + 1)
        c = [0] * (n + 1)
        d = [0] * (n + 1)
        for i in range(n):
            a[i] = λ[i]
            b[n - i - 1] = μ[i]
```

```
    for i in range(n + 1):
        c[i] = a[i] + b[i]
        d[i] = min(c[i], n - i)
    result = factorial(n) // (factorial(a[0]) * factorial(b[0]))
    for i in range(1, n + 1):
        result *= factorial(d[i] - d[i - 1]) // (factorial(a[i] - a[i - 1]) * factorial(b[i] -
    return result

def permanent(poly):
    if len(poly) == 0:
        return 1
    n = len(poly)
    det = 0
    for i in range(n):
        subpoly = [row[:i] + row[i+1:] for row in poly[1:]]
        det += (-1) ** i * poly[0][i] * permanent(subpoly)
    return det

def determinant(poly):
    if len(poly) == 0:
        return 1
    n = len(poly)
    det = 0
    for i in range(n):
        subpoly = [row[:i] + row[i+1:] for row in poly[1:]]
        det += (-1) ** i * poly[0][i] * determinant(subpoly)
    return det

def tensor_power(poly, k):
    if k == 0:
        return [[1]]
    result = []
    for term in poly:
        new_term = [term[i] * term[j] for i in range(len(term)) for j in range(len(term))]
        result.append(new_term)
    return result

def sum_littlewood_richardson(poly, λ):
    n = len(poly)
    k = floor(n ** 1.5)
    m = floor(n ** 0.5)
    det_poly = tensor_power([[i + 1 for i in range(m)]], k)
    perm_poly = tensor_power([[i + 1 for i in range(n)]], k)
    det_sum = sum(littlewood_richardson_coefficient(λ, μ) * determinant(det_poly) for μ in you
    perm_sum = sum(littlewood_richardson_coefficient(λ, μ) * permanent(perm_poly) for μ in you
    return det_sum / perm_sum

n = random.randint(5, 40)
λ = [random.randint(1, n) for _ in range(n)]
ratio = sum_littlewood_richardson(λ, λ)

conjecture_holds = ratio <= n ** (n - 1) and ratio >= n ** n
counterexample = "" if conjecture_holds else f"Ratio {ratio} does not match expected bounds"

return {
    "metric_name": "Littlewood-Richardson Ratio",
```

```
        "metric_value": ratio,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] for r in results) / len(results)
    std_ratio = (sum((r["metric_value"] - mean_ratio) ** 2 for r in results) / len(results)) ** 0.
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        result = f"RESULT: SUPPORTED mean={mean_ratio:.2f} std={std_ratio:.2f} support_fraction={s
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        result = f"RESULT: FALSIFIED counterexample=\"Ratio out of bounds\" first_failing_seed={fi
    else:
        result = "RESULT: INCONCLUSIVE mapping_undefined"

    print(result)
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test timed out before producing results, preventing verification of growth rates. Pre-registered support condition cannot be evaluated without data. | next: Optimize/test with higher time limits and larger n/k values to resolve timeout

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	42669
2	preregistration	ollama_remote	qwen3:8b	0	0	27920
3	novelty	ollama_remote	qwen3:8b	0	0	22952
4	novelty	ollama_remote	qwen3:8b	0	0	14961
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	47111
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14705
7	judge	ollama_remote	qwen3:8b	0	0	97618

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 267936 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/52ba9a0b4010.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/52ba9a0b4010.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/52ba9a0b4010.tar.gz` (if generated)